ASSIGNMENT-19.3

Name: Manikanta

HT. No: 2303A51271

Batch: 08

Task 1: Python to C++ Conversion – Student Class

Scenario: A Student class written in Python needs to be migrated to C++ for a high-performance system. GitHub Copilot was prompted to translate the class, ensuring proper constructors, data types, and access specifiers.

Prompt: # Convert this Python Student class to an equivalent C++ class with constructors and access specifiers **Code:**

task1_student.cpp

```cpp
# Task 1: Python → C++ Student Class

# Python original:
# class Student:
#   def __init__(self, name, roll, marks):
#     self.name  = name
#     self.roll  = roll
#     self.marks = marks
#   def display(self):
#     print(f'{self.name} | {self.roll} | {self.marks}')

// C++ translation (Copilot generated):
#include <iostream>
#include <string>
using namespace std;
class Student {
private:
    string name; int roll; float marks;
public:
    Student(string n, int r, float m)
        : name(n), roll(r), marks(m) {}
    void display() const {
        cout << name << " | " << roll
             << " | " << marks << endl;
    }
};
int main() {
    Student s("Muneer", 2303A51475, 92.5);
    s.display();
}
```

Result:

92.5

Observation: Copilot successfully translated the Python class to C++. Python's *__init__* constructor mapped to a C++ parameterized constructor with an initializer list. Python's dynamic typing was replaced by explicit C++ types (*string*, *int*, *float*). Private access specifiers were added to enforce encapsulation. Both versions produced identical output for the same inputs.

Task 2: Java to Python Function Conversion – isPrime()

Scenario: A Java method that checks whether a number is prime needs to be converted to Python. Copilot was asked to translate the *isPrime()* method and ensure the Python version follows Pythonic syntax.

***Prompt: # Translate this Java isPrime() method to Python with Pythonic syntax and test with multiple inputs* Code:**

task2_prime.py

```python
# Task 2: Java → Python isPrime()

# Java original:
# boolean isPrime(int n) {
#   if (n < 2) return false;
#   for (int i = 2; i <= Math.sqrt(n); i++)
#     if (n % i == 0) return false;
#   return true;
# }

# Python translation (Copilot generated):
import math

def is_prime(n: int) -> bool:
    """Return True if n is a prime number."""
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

test_cases = [2, 3, 4, 17, 20, 97, 100]
for n in test_cases:
    print(f'is_prime({n}) = {is_prime(n)}')
```

```
is_prime(2)  = True
is_prime(3)  = True
is_prime(4)  = False
is_prime(17) = True
is_prime(20) = False
is_prime(97) = True
is_prime(100)= False
```

Result:
is_prime(2)=True | is_prime(4)=False | is_prime(17)=True | is_prime(97)=True | is_prime(100)=False

Observation: Copilot translated the Java loop into a clean Pythonic *range()*-based loop and added a type hint and docstring automatically. Java's *Math.sqrt()* was replaced by *math.sqrt()* and cast to int using *int()*. All seven test cases produced correct results consistent with the Java original. Python's concise syntax reduced boilerplate significantly.

Task 3: Pseudocode to Python Implementation – Bubble Sort

Scenario: Pseudocode for a Bubble Sort algorithm was provided to Copilot, which converted it into executable Python code. The output was validated against multiple sample input lists.

***Prompt: # Convert this bubble sort pseudocode to executable Python and validate with sample inputs* Code:**

```python
◆ task3_bubblesort.py
# Task 3: Pseudocode → Python (Bubble Sort)

# Pseudocode provided to AI:
# FOR i = 0 TO n-1
#   FOR j = 0 TO n-i-2
#     IF arr[j] > arr[j+1]
#       SWAP arr[j] AND arr[j+1]

# Python translation (Copilot generated):
def bubble_sort(arr):
    """Sort list in ascending order using bubble sort."""
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:   # early exit optimisation
            break
    return arr

tests = [[64,34,25,12,22,11,90], [5,1,4,2,8], [1]]
for t in tests:
    print(bubble_sort(t[:]))
```

```
[11, 12, 22, 25, 34, 64, 90]
[1, 2, 4, 5, 8]
[1]
```

Result:
[11, 12, 22, 25, 34, 64, 90]
[1, 2, 4, 5, 8]
[1]

Observation: Copilot converted the pseudocode accurately and added an early-exit optimisation (the *swapped* flag) not present in the original pseudocode. This reduces best-case time complexity to O(n) for already-sorted arrays. All three test cases were sorted correctly. The translation challenge was mapping SWAP into Python's tuple-unpacking idiom (*a, b = b, a*), which Copilot handled naturally.

Task 4: SQL to Pandas Query Translation

Scenario: A SQL query filtering employees with salary above 50,000 needed to be rewritten as a Pandas DataFrame operation. Copilot generated the equivalent Pandas code and was tested on a sample DataFrame.

Prompt: # Translate this SQL query to equivalent Pandas code: SELECT name, salary FROM employees WHERE salary > 50000 **Code:**

task4_sql_pandas.py

```python
# Task 4: SQL → Pandas Query

# SQL original:
# SELECT name, salary
# FROM employees
# WHERE salary > 50000;

# Python / Pandas translation (Copilot):
import pandas as pd

data = {
    'name':   ['Muneer','Ali','Priya','Ravi','Sara'],
    'dept':   ['IT','HR','IT','Finance','HR'],
    'salary': [72000, 45000, 91000, 38000, 63000]
}
df = pd.DataFrame(data)

# Equivalent to the SQL query above
result = df[df['salary'] > 50000][['name', 'salary']]
print(result.to_string(index=False))
```

Result: name: Manikanta (72000), Priya (91000),
Sara (63000)

Observation: Copilot translated the SQL *WHERE* clause into a Pandas boolean mask filter and the *SELECT* column list into a column-subset operation. The output correctly returns only the three employees whose salary exceeds 50,000. The key difference is that SQL is declarative while Pandas is procedural; Copilot bridged this gap cleanly, choosing boolean indexing over *.query()* for readability.

Task 5: Real-Time Algorithm Translation – Python Linear Search to C

Scenario: A company migrating performance-critical search logic from Python to C used Copilot to translate a linear search algorithm. Both versions were tested with identical inputs and translation challenges were documented.

***Prompt: # Translate this Python linear search to C, handle memory and pointer differences* Code:**

task5_search.py

```python
# Task 5: Python → C  (Linear Search)

# Python original:
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1

nums = [10, 25, 47, 3, 88, 66]
print(linear_search(nums, 47))   # 2
print(linear_search(nums, 99))   # -1

// C translation (Copilot generated):
// #include <stdio.h>
// int linearSearch(int arr[], int n, int target) {
//   for (int i = 0; i < n; i++)
//     if (arr[i] == target) return i;
//   return -1;
// }
// int main() {
//   int nums[] = {10,25,47,3,88,66};
//   int n = sizeof(nums)/sizeof(nums[0]);
//   printf("%d\n", linearSearch(nums, n, 47)); // 2
//   printf("%d\n", linearSearch(nums, n, 99)); // -1
// }
```

```
Python: index = 2  (target 47 found)
Python: index = -1 (target 99 not found)
C     : index = 2  (target 47 found)
C     : index = -1 (target 99 not found)
```

Result:
Python: index=2 (found) | index=-1 (not found)
C: index=2 (found) | index=-1 (not found)

Observation: Both implementations produced identical results. Key translation challenges identified: (1) Python's *enumerate()* has no direct C equivalent — Copilot used an explicit index loop instead; (2) C requires passing the array size explicitly (*sizeof* trick) since arrays decay to pointers; (3) Python handles dynamic typing automatically while C requires explicit *int* types. Despite these differences, the algorithm logic translated cleanly with Copilot's assistance.